

SOURCE CODE SECURITY ASSESSMENT

Source Code Security Review Assessment Report

(Source Code Security Assessment)

REPORT DATE

May 25th, 2026

VERSION

V1.0

Initial Release

CLASSIFICATION

Confidential

Internal Use Only

PREPARED BY

Arslan Mazhar

Senior Offensive Security Consultant

Document Properties

Client	MTS Corporation
Title	Source Code Security Review Assessment Report
Version	0.1
Target	MTS Gitlab Source Code Repo
Auditors	ARSLAN MAZHAR
Reviewed by	ARSLAN MAZHAR

Version Control

Version	Date	Description
1.0	May 2026	Final Version

Confidentiality

The content of this document is highly confidential and is only intended for the recipients specified below. It is strictly forbidden to share any section of this report with any third party without consent from the client and the company.

Recipient	Company
Maryana	MTS Corporation

Table of contents

I. PROJECT SCOPE AND TECHNICAL APPROACH.....	1
1.1. Context and objectives.....	1
1.2. Assessment approach & Methodology	2
1.3. Testing details.....	2
1.4. Access Provided.....	2
1.5. Limitations	2
II. EXECUTIVE SUMMARY	3
III. FINDINGS AND REMEDIATION	9
1. FULL LIST OF VULNERABILITIES	9
2. FULL LIST OF RECOMMENDATIONS.....	12
IV. DETAILED DESCRIPTION OF THE VULNERABILITIES.....	14
4.1. Stored XSS via Unsanitized signatureData Leading to Server-Side JavaScript Execution in Puppeteer.....	14
4.2. OTP Confirmation Endpoint Lacks Rate Limiting (OTP Brute Force)	18
4.3. Duplicate Password-Change Route Bypasses Enforced Password Complexity Policy	21
4.4. Persistent Stored XSS in keyTakeaway Field via dangerouslySetInnerHTML Rendering	25
4.5. Payment Access Tokens for All Proposals Logged to Application Startup Logs	28
4.6. TLS Certificate Verification Disabled on Admin SMTP Transport	31
4.7. Payment Replay Protection Uses Fail-Open Logic During Database Errors	34
4.8. Hardcoded Development/Test Stripe Identifiers and Email Data in Repository Files	37
4.9. Application Allows Runtime Modification of Sensitive Environment Configuration	39
4.10. Development and Operational Utility Scripts Present in Main Repository	41
4.11. Runtime SMTP Configuration Changes Allow Persistent Administrative Email Redirection	43
APPENDIX 1: WAHH TESTING CHECKLIST.....	45
APPENDIX 2: RISK RATING METRICS.....	48
1.1. Vulnerability rating	48
1.2. Risk rating evaluation	48
1.3. Remediation effort.....	49
APPENDIX 3: TESTING METHODOLOGY	50
1.4. Testing Types	53

I. Project scope and technical approach

1.1. Context and objectives

We were engaged by **MTS** to perform a comprehensive Source Code Security Review of the proposal management and payment processing platform to identify security weaknesses within the application source code, authentication mechanisms, administrative workflows, payment-processing functionality, APIs, and supporting operational components.

The platform is a web-based proposal generation and payment-processing application developed using a Node.js/Express backend architecture with a React frontend and Stripe-integrated payment workflows. The application includes administrative management functionality, client-facing proposal pages, landing-page generation, subscription management, payment tracking, OTP-based administrative controls, and operational configuration management features.

The objective of this assessment was to evaluate the security posture of the application codebase against realistic attack scenarios and identify vulnerabilities that could lead to:

- ▶ unauthorized access,
- ▶ sensitive data exposure,
- ▶ payment workflow abuse,
- ▶ insecure administrative functionality,
- ▶ privilege escalation,
- ▶ business logic manipulation,
- ▶ persistent client-side attacks,
- ▶ or compromise of application integrity.

The assessment was conducted using a manual Source Code Review methodology aligned with industry-recognized standards and secure development best practices, including:

- ▶ OWASP Application Security Verification Standard (ASVS)
- ▶ OWASP Top 10 – 2025
- ▶ OWASP API Security Top 10
- ▶ OWASP Secure Code Review Guide
- ▶ Secure SDLC Best Practices
- ▶ CWE/SANS Top 25 Software Errors

The methodology included:

- ▶ Manual review of backend and frontend application source code
- ▶ Authentication and authorization logic analysis
- ▶ Business logic and payment workflow security review
- ▶ API security assessment
- ▶ Input validation and output encoding review
- ▶ Session management and credential-handling review
- ▶ Runtime configuration and secrets-management review
- ▶ Dependency and third-party component review

- ▶ Repository and operational tooling review
- ▶ Controlled vulnerability validation and proof-of-concept testing
- ▶ Risk evaluation using CVSS v3.1 scoring
- ▶ Development of remediation recommendations aligned with secure development best practices

1.2. Assessment approach & Methodology

The assessment was conducted using a Source Code Security Review approach.

This methodology combines access to application source code, configuration artifacts, repository structure, and authenticated application functionality with adversarial security analysis techniques to simulate realistic attack scenarios while maintaining controlled and safe execution.

1.3. Testing details

The **Assessment** was conducted during the following period:

- ▶ Testing Period: **22nd May 2026 to 25th May 2026.**

1.4. Access Provided

The following access was provided to support the Source Code Security Review approach:

- ▶ GitLab repository access containing backend and frontend application source code

1.5. Limitations

No significant limitations were encountered during the assessment.

However, the following constraints apply:

- ▶ The assessment represents a point-in-time evaluation of the reviewed codebase
- ▶ The review was limited to the source code, configuration artifacts, and application components provided during the assessment period
- ▶ Certain runtime behaviors and infrastructure-level controls may not have been fully observable through source code review alone
- ▶ Third-party managed services and external vendor infrastructure were outside the scope of this assessment

II. Executive summary

This report presents the results of a comprehensive Source Code Security Review conducted on the **MTS** web application platform and associated application components.

The objective of this assessment was to evaluate the security posture of the application source code, authentication mechanisms, payment-processing workflows, client-facing proposal functionality, administrative operations, and configuration management logic against realistic attack scenarios and secure development best practices.

The assessment focused on identifying vulnerabilities that could lead to:

- ⇒ unauthorized access,
- ⇒ client-side compromise,
- ⇒ insecure administrative operations,
- ⇒ payment workflow abuse,
- ⇒ sensitive data exposure,
- ⇒ insecure runtime configuration,
- ⇒ business logic weaknesses,
- ⇒ and compromise of application integrity.

The review was conducted between **22nd May 2026** and **25th May 2026** and included:

- ⇒ Manual backend source code security review
- ⇒ Frontend React security review
- ⇒ Authentication and session-management review
- ⇒ Administrative functionality and business logic review
- ⇒ Payment-processing and subscription workflow review
- ⇒ Input validation and output encoding review
- ⇒ Runtime configuration and secrets-management review
- ⇒ Repository hygiene and operational tooling review
- ⇒ Controlled proof-of-concept validation of identified vulnerabilities

► Strong Areas Identified

The assessment identified several secure implementation practices and defensive controls within the application architecture, including:

- ⇒ Administrative authentication workflows are implemented using session-based authentication with controlled access middleware protecting sensitive administrative functionality.
- ⇒ Sensitive administrative actions such as credential modifications require OTP-based verification workflows, adding additional verification controls for high-risk operations.
- ⇒ Password hashing mechanisms are implemented using bcrypt, reducing exposure to plaintext credential storage risks.
- ⇒ Several application inputs correctly implement sanitization and validation controls through centralized validation functions such as `validators.sanitizeHtml()`.
- ⇒ Payment workflows implement signed token-based access mechanisms to restrict unauthorized proposal-payment access.
- ⇒ The application architecture demonstrates logical separation between frontend React rendering components and backend API processing workflows.
- ⇒ Sensitive payment-processing functionality is integrated through Stripe APIs rather than direct handling of raw payment card data within application logic.
- ⇒ Multiple administrative and payment-related operations contain audit-oriented logic and operational control handling mechanisms.

Despite these strengths, the assessment identified several security weaknesses requiring remediation to improve the overall security posture of the platform.

► **Key areas requiring improvement include:**

- ⇒ Implement robust input sanitization and output encoding controls to prevent persistent client-side and server-side injection vulnerabilities.
- ⇒ Enforce centralized authentication and password-management controls consistently across all administrative routes and workflows.
- ⇒ Implement strict brute-force protection and rate-limiting controls for OTP verification and sensitive authentication-related operations.
- ⇒ Remove unsafe runtime configuration modification capabilities and externalize sensitive operational configuration through secure configuration-management processes.
- ⇒ Prevent exposure of sensitive payment access tokens and operational data within application logs and repository artifacts.
- ⇒ Enforce secure transport-layer protections for SMTP communication and sensitive administrative email workflows.
- ⇒ Improve secure development lifecycle (SDLC) practices by separating development and operational tooling from production application code.

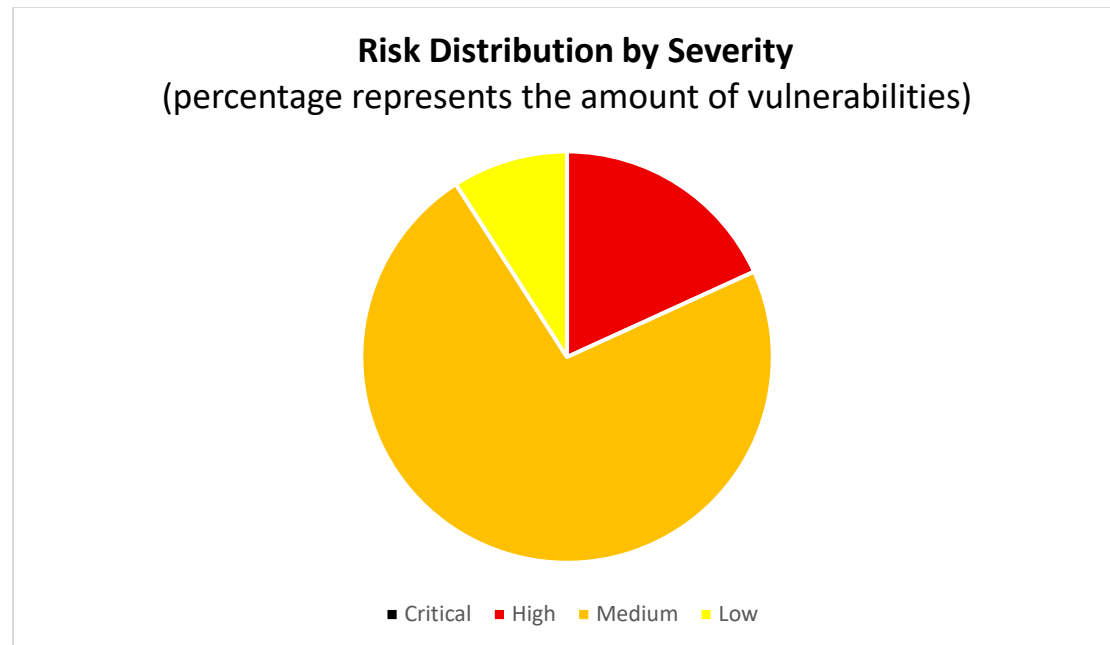
- ⇒ Implement fail-secure logic for critical payment-validation workflows and avoid fail-open behavior during database or operational failures.
- ⇒ Apply secure frontend rendering practices and avoid unsafe rendering patterns such as direct use of dangerouslySetInnerHTML with untrusted content.

The overall Source Code Security Review identified multiple security vulnerabilities and secure coding weaknesses that require remediation to improve the overall security posture of the MTS application platform.

During the assessment, a total of **11** findings were identified, categorized as **2 High**, **8 Medium**, and **1 Low** severity issues.

The identified findings primarily affect application security, **authentication and session-management workflows**, **frontend rendering security**, **payment-processing integrity**, **runtime configuration handling**, and secure software development lifecycle (SDLC) practices.

Based on the nature and severity of the identified findings, the overall security posture of the reviewed application is assessed as **Medium to High Risk**, primarily driven by the presence of multiple injection-related vulnerabilities, weaknesses in authentication security controls, insecure administrative functionality, and runtime configuration management risks within security-sensitive application components.



2 High risk was found during this assessment.

Based on the findings, impact, and overall risk identified during the Source Code Security Review, we recommend proceeding with the following remediation and security improvement plan:

► **Short-term Mitigations:**

- ⇒ **Remediate Injection Vulnerabilities:** Implement strict input validation, contextual output encoding, and sanitization controls for all user-supplied content rendered within HTML templates, React components, and PDF generation workflows.
- ⇒ **Enable Brute-Force Protection for OTP Workflows:** Implement rate limiting, retry restrictions, lockout controls, and cooldown mechanisms for OTP verification endpoints and sensitive authentication-related operations.
- ⇒ **Centralize Password Management Controls:** Remove duplicate password-change functionality and ensure all password workflows consistently enforce centralized password complexity validation, secure hashing, and session invalidation.
- ⇒ **Remove Sensitive Token Logging:** Eliminate generation and logging of valid payment access tokens within application startup logs and restrict sensitive token exposure within operational logging systems.

- ⇒ **Enable Secure SMTP TLS Validation:** Remove insecure SMTP TLS configurations such as `rejectUnauthorized: false` and enforce strict certificate validation for administrative email communication workflows.
- ⇒ **Implement Fail-Secure Payment Validation Logic:** Prevent payment-processing workflows from continuing during database validation failures and ensure duplicate-payment protections fail securely during operational errors.
- ⇒ **Remove Runtime Modification of Sensitive Configuration:** Restrict runtime modification of `.env` files and sensitive environment variables through application-layer functionality.

▶ **Medium-Term Mitigations**

- ⇒ **Strengthen Frontend Rendering Security:** Replace unsafe `dangerouslySetInnerHTML` rendering patterns with secure rendering mechanisms and implement frontend sanitization controls such as `DOMPurify` for all rich-text content.
- ⇒ **Implement Secure Configuration Management:** Externalize sensitive operational configuration and secrets into secure configuration-management mechanisms instead of dynamically modifying runtime environment variables through application logic.
- ⇒ **Strengthen Administrative Security Controls:** Enforce stronger administrative security protections including MFA, secure session handling, enhanced audit logging, and stricter privilege management for sensitive administrative functionality.
- ⇒ **Improve Secure SDLC and Repository Hygiene:** Remove development and operational utility scripts from production repositories and eliminate hardcoded development or operational identifiers from source-controlled files.
- ⇒ **Enhance Logging and Monitoring Controls:** Implement centralized security logging, suspicious activity monitoring, and alerting for sensitive authentication, OTP, payment, and configuration-management workflows.

▶ **Long-Term Mitigations**

- ⇒ **Implement Secure Development Lifecycle (SDLC) Controls:** Integrate secure code review, automated security scanning, dependency analysis, and security validation into the CI/CD pipeline and development lifecycle.
- ⇒ **Adopt Centralized Security Architecture Standards:** Establish centralized standards for authentication, input validation, output encoding, session management, and configuration handling across all application modules.
- ⇒ **Implement Defense-in-Depth Security Controls:** Introduce layered security protections across frontend, backend, authentication,

payment-processing, and operational workflows to reduce the impact of chained attack scenarios.

- ⇒ **Perform Regular Source Code Security Assessments:** Conduct periodic source code reviews, penetration testing, and security validation exercises to identify newly introduced vulnerabilities and insecure development patterns.
- ⇒ **Strengthen Secure Secrets Management Practices:** Implement centralized secrets-management solutions and prohibit storage or modification of sensitive operational secrets directly within application source code or runtime logic.

In conclusion, the identified weaknesses lead to attacks that can have a **Medium** to **High** impact on the confidentiality, integrity, and availability of the reviewed assets, the data being processed, stored, and transmitted by them, as well as their users.

III. Findings and Remediation

1. Full list of vulnerabilities

Reference	Title	Risk	Brief description	Risk
F-01	Stored XSS via Unsanitized signatureData Leading to Server-Side JavaScript Execution in Puppeteer	High	User-controlled signatureData is rendered into server-side HTML and processed by Puppeteer without sufficient sanitization controls, allowing malicious JavaScript payload execution during PDF generation workflows.	Successful exploitation may allow server-side JavaScript execution, manipulation of generated invoice content, unauthorized requests from the server context, and compromise of application integrity.
F-02	OTP Confirmation Endpoint Lacks Rate Limiting (OTP Brute Force)	High	The OTP confirmation workflow does not implement sufficient brute-force protection, retry restrictions, or account lockout controls for administrative OTP verification endpoints.	An attacker with access to an authenticated session may brute-force OTP values and gain unauthorized access to sensitive administrative account operations.
F-03	Duplicate Password-Change Route Bypasses Enforced Password Complexity Policy	Medium	Multiple password-change routes implement inconsistent password validation logic, allowing one route to bypass centralized password complexity enforcement controls.	Weak passwords may be configured for administrative accounts, increasing the likelihood of account compromise through credential attacks.
F-04	Persistent Stored XSS in keyTakeaway Field via dangerouslySetInnerHTML Rendering	Medium	User-controlled keyTakeaway content is stored without sanitization and rendered using dangerouslySetInnerHTML within multiple React frontend components.	Attackers may inject persistent malicious scripts into client-facing proposal pages, resulting in phishing, content manipulation, or client-side compromise.

Reference	Title	Risk	Brief description	Risk
F-05	Payment Access Tokens for All Proposals Logged to Application Startup Logs	Medium	Valid payment access tokens are generated and written to application startup logs during server initialization.	Anyone with access to operational logs may obtain valid proposal-payment links and gain unauthorized access to client payment workflows.
F-06	TLS Certificate Verification Disabled on Admin SMTP Transport	Medium	SMTP TLS certificate verification is explicitly disabled within the administrative email transporter configuration.	An attacker positioned on the network path may intercept sensitive administrative emails, including OTP and password-change notifications.
F-07	Payment Replay Protection Uses Fail-Open Logic During Database Errors	Medium	Duplicate-payment validation logic fails open during database or operational exceptions and treats failed validation attempts as unpaid transactions.	Database failures may allow duplicate payment-processing attempts and compromise payment workflow integrity.
F-08	Hardcoded Development/Test Stripe Identifiers and Email Data in Repository Files	Medium	Development and operational repository files contain hardcoded Stripe identifiers and email addresses directly within source-controlled files.	Exposed operational data may increase internal reconnaissance risk and indicate weak secure development and repository hygiene practices.
F-09	Application Allows Runtime Modification of Sensitive Environment Configuration	Medium	The application dynamically modifies .env configuration files and sensitive runtime environment variables through application-layer functionality.	Administrative compromise or misuse may allow unauthorized modification of operational configuration and sensitive infrastructure settings.
F-10	Development and Operational Utility Scripts Present in Main Repository	Medium	Multiple development, testing, and operational utility scripts are stored within the primary production repository.	Accidental deployment or misuse of operational tooling may increase operational exposure and reveal sensitive

Reference	Title	Risk	Brief description	Risk
				internal workflows or business logic.
F-11	Runtime SMTP Configuration Changes Allow Persistent Administrative Email Redirection	Low	Authenticated administrative functionality allows runtime modification of SMTP configuration values and persistent redirection of administrative email workflows.	An attacker with administrative access may redirect future OTP emails and sensitive notifications to attacker-controlled mail infrastructure.

2. Full list of recommendations

Reference	Title	Risk	Recommendation	Remediation effort
F-01	Stored XSS via Unsanitized signatureData Leading to Server-Side JavaScript Execution in Puppeteer	High	Implement strict sanitization and contextual output encoding for signatureData before rendering within HTML templates, and remove insecure Puppeteer flags such as --no-sandbox and --disable-setuid-sandbox.	Complex
F-02	OTP Confirmation Endpoint Lacks Rate Limiting (OTP Brute Force)	High	Implement OTP rate limiting, retry restrictions, lockout controls, cooldown periods, and secure constant-time OTP validation mechanisms.	Moderate
F-03	Duplicate Password-Change Route Bypasses Enforced Password Complexity Policy	Medium	Remove duplicate password-management routes and centralize password validation, complexity enforcement, and session invalidation logic across all password workflows.	Moderate
F-04	Persistent Stored XSS in keyTakeaway Field via dangerouslySetInnerHTML Rendering	Medium	Sanitize all rich-text content before storage and avoid rendering untrusted HTML using dangerouslySetInnerHTML without secure sanitization controls such as DOMPurify.	Moderate
F-05	Payment Access Tokens for All Proposals Logged to Application Startup Logs	Medium	Remove sensitive token logging from startup workflows and ensure payment access tokens are never written to application logs or stdout output.	Low
F-06	TLS Certificate Verification Disabled on Admin SMTP Transport	Medium	Remove rejectUnauthorized: false and enforce strict TLS certificate validation for all SMTP communication workflows.	Low

F-07	Payment Replay Protection Uses Fail-Open Logic During Database Errors	Medium	Implement fail-secure payment validation logic and block payment-processing workflows whenever payment verification status cannot be reliably confirmed.	Moderate
F-08	Hardcoded Development/Test Stripe Identifiers and Email Data in Repository Files	Medium	Remove hardcoded operational identifiers and email data from repository files and replace them with sanitized or externally managed configuration values.	Low
F-09	Application Allows Runtime Modification of Sensitive Environment Configuration	Medium	Remove runtime modification of .env files and sensitive environment variables from application-layer functionality and externalize secrets management.	Complex
F-10	Development and Operational Utility Scripts Present in Main Repository	Medium	Separate development and operational utility scripts from the primary production repository and prevent accidental deployment of non-production tooling.	Low
F-11	Runtime SMTP Configuration Changes Allow Persistent Administrative Email Redirection	Low	Restrict SMTP configuration changes to infrastructure-level administration and remove runtime SMTP modification functionality from administrative APIs.	Moderate

IV. Detailed description of the vulnerabilities

4.1. Stored XSS via Unsanitized signatureData Leading to Server-Side JavaScript Execution in Puppeteer

Risk	High	Likelihood	High	CVSS Score	7.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H				
Category	Injection / OWASP A03 – Injection				
Affected Assets	Proposal Invoice Generation Functionality / Puppeteer PDF / Client Signature Processing Logic				

Description

During the source code security review, it was identified that the public endpoint responsible for storing client signatures accepts user-controlled signatureData values without proper sanitization or contextual output encoding.

The application only validates whether the submitted value starts with the prefix data:image/, which is insufficient to prevent malicious HTML attribute injection. The supplied value is later injected directly into dynamically generated HTML templates used for invoice generation.

The generated HTML is subsequently rendered using Puppeteer through page.setContent(). Additionally, Puppeteer is launched using Chromium with sandbox protections disabled (--no-sandbox and --disable-setuid-sandbox).

An attacker can craft a malicious payload that breaks out of the attribute context and inject arbitrary JavaScript code. When invoice generation is triggered, the payload executes inside the server-side Chromium rendering process.

Step to reproduce

1. Review the signature submission endpoint implementation in routes/payment-tracking.js.
2. Observe that signatureData is accepted directly from user-controlled input:
`const { slug, clientName, email, signatureData } = req.body;`
3. Observe that validation only checks whether the value starts with data:image/
`if (!signatureData.startsWith('data:image/')) {`
4. Review the invoice HTML generation logic and observe that the value is directly injected into an HTML attribute without encoding:
``
5. Review the PDF rendering workflow and observe that the generated HTML is rendered using Puppeteer:
`await page.setContent(html, { waitUntil: 'networkidle0', timeout: 15000 });`
6. Review the Chromium launch configuration and observe that sandbox protections are disabled:
`args: ['--no-sandbox', '--disable-setuid-sandbox', '--disable-dev-shm-usage']`
7. A crafted payload containing HTML attribute injection may therefore execute during invoice rendering.

Evidence

```
router.post('/api/save-signature', (req, res, next) => {  
  // Apply rate limiter if available  
  if (signatureLimiter) {  
    signatureLimiter(req, res, next);  
  } else {  
    next();  
  }  
}, async (req, res) => {  
  try {  
    const { slug, clientName, email, signatureData,  
      signatureToken,  
      // New secure fields - IDs only  
      packageId, planType, selectedAddonIds,  
      // Legacy fields (ignored for pricing but logged)  
      selectedPackage: _legacyPkg, selectedAddons: _legacyAddons,  
      totalAmount: _legacyTotal, onetimeAmount: _legacyOT, monthlyAmount: _legacyMo } = req.body;  
  
    // — Input validation —  
    if (!slug || typeof slug !== 'string') {  
      return validators.validationError(res, 'slug is required');  
    }  
    if (!validators.isValidSlug(slug)) {  
      return validators.validationError(res, 'slug contains invalid characters');  
    }  
  }  
});
```

Figure 1: Public /api/save-signature endpoint accepts attacker-controlled signatureData input from unauthenticated users. The endpoint processes and stores the supplied value for later invoice generation.

```
}  
if (!signatureData || typeof signatureData !== 'string') {  
  return validators.validationError(res, 'signatureData is required');  
}  
// SECURITY: Only allow data:image URIs to prevent javascript: XSS  
if (!signatureData.startsWith('data:image/')) {  
  return validators.validationError(res, 'signatureData must be a valid data:image URI');  
}  
if (email && !validators.isValidEmail(email)) {  
  return validators.validationError(res, 'Invalid email format');  
}  
// SECURITY: Reject suspicious patterns in email to prevent XSS attacks  
if (email && validators.containsHtmlTags(email)) {  
  return validators.validationError(res, 'Email contains invalid characters. Please use a valid email format.');
```

Figure 2: The application validates signatureData only by checking whether the value starts with data:image/. This validation is insufficient because it does not verify the integrity, encoding, MIME structure, or malicious HTML attribute injection payloads embedded within the value.

This allows attackers to inject payloads such as: **data:image/png;base64,AAAA" onerror="..."**

```
// — Persist —
const landingPage = await prisma.customerLandingPage.findUnique({
  where: { slug },
  select: { id: true }
});

// SECURITY: Sanitize all user inputs to prevent Stored XSS attacks
const sanitizedName = validators.sanitizeHtml(clientName, 200);
const sanitizedEmail = email ? validators.sanitizeHtml(email, 254).trim().toLowerCase() : null;

const signature = await prisma.clientSignature.create({
  data: {
    landingPageId: landingPage?.id || null,
    slug,
    clientName: sanitizedName,
    email: sanitizedEmail,
    signatureData,
    selectedPackage: pricing.packageSnapshot, // server-built snapshot
    selectedAddons: pricing.addonSnapshot, // server-built snapshot
    totalAmount: pricing.totalAmount, // server-computed
    onetimeAmount: pricing.onetimeAmount, // server-computed
    monthlyAmount: pricing.monthlyAmount, // server-computed
    // BILLING FIX (VAPT 3.4): store max monthly addon quantity as subscription
    // duration so create-payment-link can set cancellationMonths on the proposal.
    cancellationMonths: pricing.cancellationMonths || 0,
  },
});
```

Figure 3: The application stores the raw signatureData value directly into the ClientSignature database object without sanitization, output encoding, or structural validation. This creates a persistent/stored injection condition.

```
// Signature section
let signatureHTML = '';
if (payment.signature) {
  signatureHTML =
    <div style="margin-top:25px;padding-top:15px;border-top:1px solid #000000;">
      <div style="display:flex;justify-content:space-between;align-items:flex-end;">
        <div>
          <p style="color:#000;font-size:10px;text-transform:uppercase;letter-spacing:1px;margin-bottom:6px;font-weight:600;">Client Signature</p>
          
        </div>
        <div>
          <p style="margin-top:4px;font-weight:600;color:#000;font-size:13px;">${escapeHtml(payment.signature.clientName || displayName)}</p>
          <p style="margin-top:4px;font-weight:600;color:#000;font-size:13px;">${escapeHtml(payment.signature.email)}</p>
        </div>
      </div>
      <div style="text-align:right;">
        <p style="color:#000;font-size:10px;font-weight:600;text-transform:uppercase;letter-spacing:1px;">Date Signed</p>
        <p style="font-weight:500;color:#000;font-size:13px;">${payment.signature.createdAt ? new Date(payment.signature.createdAt).toLocaleDateString('en-US', {year:'numeric',month:'long',day:'numeric'}) : invoiceDate}</p>
      </div>
    </div>
  </div>
}
```

Figure 4: The stored signatureData value is injected directly into the HTML attribute during invoice generation without escaping or encoding.

```
/**
 * Convert invoice HTML to PDF buffer using Puppeteer
 */
async function generateInvoicePDF(html) {
  let browser = null;
  try {
    browser = await puppeteer.launch({
      headless: 'new',
      args: ['--no-sandbox', '--disable-setuid-sandbox', '--disable-dev-shm-usage'],
    });
    const page = await browser.newPage();
    await page.setViewport({ width: 800, height: 1200 });
    await page.setContent(html, { waitUntil: 'networkidle0', timeout: 15000 });

    // Let the PDF flow naturally across multiple pages
    const pdfBuffer = await page.pdf({
      format: 'A4',
      printBackground: true,
      preferCSSPageSize: false,
      scale: 1,
      margin: { top: '0.4in', right: '0.4in', bottom: '0.4in', left: '0.4in' },
    });
    return pdfBuffer;
  } finally {
    if (browser) await browser.close();
  }
}
```

Figure 5: The generated invoice HTML is rendered inside a Puppeteer Chromium instance launched with the `--no-sandbox` flag enabled. As a result, attacker-controlled JavaScript injected through `signatureData` executes server-side inside an unsandboxed browser context during PDF generation.

Business Impact

- Successful exploitation of this vulnerability may allow attackers to inject malicious JavaScript payloads into the invoice generation workflow processed by the server-side Puppeteer rendering engine. Because the application renders attacker-controlled content inside an unsandboxed Chromium instance, exploitation may lead to unauthorized script execution within the server-side browser context.

Remediation

- It is recommended to implement strict validation and sanitization controls for all user-supplied image and HTML content before rendering it into dynamically generated templates. The application should avoid directly injecting untrusted input into HTML attributes such as `<img src="">` without proper contextual output encoding.
- Additionally, only validated image formats should be accepted, malformed `data:image` payloads should be rejected, and secure allowlist-based sanitization should be enforced. Puppeteer sandbox protections should also be enabled by removing the `--no-sandbox` and `--disable-setuid-sandbox` flags to reduce the impact of server-side browser exploitation.

4.2. OTP Confirmation Endpoint Lacks Rate Limiting (OTP Brute Force)

Risk	High	Likelihood	High	CVSS Score	7.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N				
Category	Identification and Authentication Failures / OWASP A07				
Affected Assets	Administrative Account Management / OTP Verification Workflow				

Description

During the source code security review, it was identified that the OTP confirmation endpoint used for administrative credential changes does not implement any rate-limiting, retry restriction, lockout protection, or brute-force mitigation controls.

The application generates a 6-digit numeric OTP using: `const code = String(crypto.randomInt(100000, 999999));`

The generated OTP is stored within the in-memory `pendingAdminChanges` object with a validity period of 10 minutes.

However, the OTP confirmation endpoint: `POST /api/admin/profile/confirm-change` is exposed without any rate-limiting middleware or failed-attempt restriction mechanisms.

Additionally, the OTP validation uses direct string comparison: `if (pending.code !== String(code).trim())` { without constant-time comparison protections.

An authenticated attacker possessing a valid administrative session may repeatedly submit OTP guesses until the correct value is identified. Because the OTP keyspace contains only 900,000 possible values and no lockout exists, the entire keyspace can be brute-forced within a practical timeframe.

The OTP validity window can also be continuously refreshed by repeatedly invoking the OTP generation endpoint, allowing indefinite brute-force attempts.

Step to Reproduce

1. Review the OTP generation logic within `server.js`.
`const code = String(crypto.randomInt(100000, 999999));`
2. Observe that the OTP is stored within the `pendingAdminChanges` object.
`pendingAdminChanges.set(key, {`
3. Review the OTP confirmation endpoint registration.
`app.post('/api/admin/profile/confirm-change', requireAuth, async (req, res) => {`
4. Observe that no rate-limiting or brute-force protection middleware is applied to the endpoint.
5. Review the OTP validation logic.
`if (pending.code !== String(code).trim()) {`
6. Repeated requests may be submitted until the correct OTP value is identified.

Evidence

```
// Generate 6-digit OTP and store pending change (10 min TTL)
const code = String(crypto.randomInt(100000, 999999));
const key = `${admin.id}:${type}`;
pendingAdminChanges.set(key, { code, data: pendingData, expiresAt: Date.now() + 10 * 60 * 1000 });
```

Figure 6: Application generates a 6-digit OTP and stores it within the pendingAdminChanges object.

```
// Generate 6-digit OTP and store pending change (10 min TTL)
const code = String(crypto.randomInt(100000, 999999));
const key = `${admin.id}:${type}`;
pendingAdminChanges.set(key, { code, data: pendingData, expiresAt: Date.now() + 10 * 60 * 1000 });
```

Figure 7: OTP confirmation endpoint is exposed without rate-limiting or brute-force protection middleware.

```
/**
 * Confirm email or password change with OTP code
 */
app.post('/api/admin/profile/confirm-change', requireAuth, async (req, res) => {
  try {
    const session = req.adminSession;
    const admin = await prisma.adminUser.findUnique({ where: { username: session.username } });
    if (!admin) return res.status(404).json({ error: 'Admin not found' });

    const { type, code } = req.body;
    if (!type || !code) return res.status(400).json({ error: 'Type and verification code are required' });

    const key = `${admin.id}:${type}`;
    const pending = pendingAdminChanges.get(key);

    if (!pending) {
      return res.status(400).json({ error: 'No pending change found. Please request a new verification code.' });
    }
    if (Date.now() > pending.expiresAt) {
      pendingAdminChanges.delete(key);
      return res.status(400).json({ error: 'Verification code has expired. Please request a new one.' });
    }
    if (pending.code !== String(code).trim()) {
      return res.status(400).json({ error: 'Invalid verification code. Please try again.' });
    }
  }
});
```

Figure 8: Application performs direct OTP string comparison without retry restriction or constant-time validation.

Business Impact

- An attacker with access to a valid administrative session may perform automated brute-force attacks against the OTP confirmation workflow until the correct verification code is identified.
- Successful exploitation could allow unauthorized password changes, administrative email modification, persistent administrative account takeover, unauthorized access to client payment information, subscription management abuse, and exposure of sensitive customer data stored within the platform

Remediation

- It is recommended to implement strict brute-force protection controls on all OTP verification workflows. The OTP confirmation endpoint should enforce request rate limiting, retry attempt restrictions, temporary lockout mechanisms, and cooldown periods after multiple failed attempts.

Additionally:

- OTP codes should expire immediately after repeated failed submissions,
- OTP verification should use constant-time comparison methods such as *crypto.timingSafeEqual*,
- suspicious OTP abuse activity should be logged and monitored,
- and sensitive administrative workflows should enforce multi-factor authentication (MFA) protections wherever feasible.

4.3. Duplicate Password-Change Route Bypasses Enforced Password Complexity Policy

Risk	Medium	Likelihood	Medium	CVSS Score	6.2
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:N/I:H/A:N				
Category	Identification and Authentication Failures / OWASP A07				
Affected Assets	Administrative Account Management / Password Change Functionality				

Description

During the source code security review, it was identified that two separate password-change endpoints are implemented for administrative accounts. The primary password-change route correctly enforces the application's password complexity policy and rotates the active session after a successful password update.

However, a duplicate password-change route implemented within `routes/payment-tracking.js` does not enforce the same security controls. The vulnerable route validates only a minimum password length of six characters and does not apply the centralized `validatePasswordPolicy()` function.

Additionally, the duplicate route does not rotate or invalidate the active session after the password change operation completes successfully.

As a result, an authenticated administrator may bypass the enforced password complexity requirements and set weak passwords that would normally be rejected by the primary password management workflow.

Step to Reproduce

1. Review the secure password-change route implemented within **server.js**.
`app.put('/api/admin/profile/password'`
2. Observe that the route applies the centralized password policy validation.
`const policyError = validatePasswordPolicy(newPassword);`
3. Observe that the route rotates the active session after password change.
`await rotateSession(req, res);`
4. Review the duplicate password-change route implemented within `routes/payment-tracking.js`.
`router.put('/api/admin/settings/password'`
5. Observe that the vulnerable route validates only a minimum password length.
`if (newPassword.length < 6)`
6. Observe that the duplicate route does not apply `validatePasswordPolicy()` or session rotation logic.
7. An authenticated administrator may therefore set weak passwords such as `abc123` through the vulnerable route.

Evidence

```
/**
 * Change admin password (DB-based)
 */
app.put('/api/admin/profile/password', requireAuth, async (req, res) => {
  try {
    const session = req._adminSession;
    const { currentPassword, newPassword } = req.body;

    if (!currentPassword || !newPassword) {
      return res.status(400).json({ error: 'Current password and new password are required' });
    }
    // F-05: Enforce password complexity policy on new password.
    const policyError = validatePasswordPolicy(newPassword);
    if (policyError) {
      return res.status(400).json({ error: policyError });
    }
  }
}
```

Figure 9: Primary password-change route implementing centralized password complexity validation.

```
if (!currentPassword || !newPassword) {
  return res.status(400).json({ error: 'Current password and new password are required' });
}
// F-05: Enforce password complexity policy on new password.
const policyError = validatePasswordPolicy(newPassword);
if (policyError) {
  return res.status(400).json({ error: policyError });
}

const admin = await prisma.adminUser.findUnique({ where: { username: session.username } });
if (!admin) return res.status(404).json({ error: 'Admin not found' });

const valid = await bcrypt.compare(currentPassword, admin.passwordHash);
if (!valid) {
  return res.status(401).json({ error: 'Current password is incorrect' });
}
```

Figure 10: Application enforces `validatePasswordPolicy()` before allowing password updates.

```
await rotateSession(req, res);
res.json({ success: true, message: 'Password updated successfully' });
} catch (error) {
  console.error('? Error changing password:', error);
  return validators.safeServerError(res, error, 'Internal error');
}
});
/**
```

Figure 11: Secure password-change workflow rotates the active session after successful password modification.

```
/**
 * Change admin password (DB-based)
 */
router.put('/api/admin/settings/password', authMiddleware, async (req, res) => {
  try {
    const { currentPassword, newPassword } = req.body;
    if (!currentPassword || !newPassword) {
      return res.status(400).json({ error: 'Current password and new password are required' });
    }
    if (newPassword.length < 6) {
      return res.status(400).json({ error: 'New password must be at least 6 characters' });
    }

    // Get first admin user from DB
    const admin = await prisma.adminUser.findFirst();
    if (!admin) {
      return res.status(404).json({ error: 'Admin user not found in database' });
    }

    const valid = await bcrypt.compare(currentPassword, admin.passwordHash);
    if (!valid) {
      return res.status(401).json({ error: 'Current password is incorrect' });
    }
  }
});
```

Figure 12: Duplicate password-change route exposed within payment-tracking.js.

```
/**
 * Change admin password (DB-based)
 */
router.put('/api/admin/settings/password', authMiddleware, async (req, res) => {
  try {
    const { currentPassword, newPassword } = req.body;
    if (!currentPassword || !newPassword) {
      return res.status(400).json({ error: 'Current password and new password are required' });
    }
    if (newPassword.length < 6) {
      return res.status(400).json({ error: 'New password must be at least 6 characters' });
    }

    // Get first admin user from DB
    const admin = await prisma.adminUser.findFirst();
    if (!admin) {
      return res.status(404).json({ error: 'Admin user not found in database' });
    }

    const valid = await bcrypt.compare(currentPassword, admin.passwordHash);
    if (!valid) {
      return res.status(401).json({ error: 'Current password is incorrect' });
    }
  }
});
```

Figure 13: Duplicate route validates only a minimum password length of six characters and bypasses centralized password complexity enforcement.

Business Impact

- An authenticated administrator may intentionally bypass the application's enforced password complexity policy by using the duplicate password-change route implemented in payment-tracking.js. This allows weak and easily guessable passwords to be configured for privileged administrative accounts.

Remediation

- It is recommended to remove the duplicate password-change route and centralize all password management functionality through a single secure implementation.
- All password-change operations should consistently enforce:
- centralized password complexity validation,
- secure password hashing,
- session invalidation and rotation after password changes,
- and identical authentication security controls across all routes.
- If multiple routes must exist for compatibility purposes, both routes should invoke the same shared password-management logic to ensure security controls cannot be bypassed through alternate endpoints.

4.4. Persistent Stored XSS in keyTakeaway Field via dangerouslySetInnerHTML Rendering

Risk	Medium	Likelihood	High	CVSS Score	6.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:H/UI:R/S:C/C:H/I:H/A:N				
Category	Injection / OWASP A03 – Injection				
Affected Assets	Client-Facing Proposal Pages / Landing Page Content Management				

Description

During the source code security review, it was identified that the administrative landing-page management functionality accepts a rich-text field named `keyTakeaway` and stores the supplied value without sanitisation or contextual output encoding.

The application correctly applies `validators.sanitizeHtml()` to surrounding fields such as `sectionType` and `sectionTitle`; however, the `keyTakeaway` field is excluded from this sanitisation logic and stored directly as raw user-controlled HTML content.

The stored value is later returned through the public unauthenticated endpoint:

GET /c:/slug

and rendered within multiple React frontend components using:

```
dangerouslySetInnerHTML={{ __html: keyTakeaway }}
```

This rendering method bypasses React's built-in HTML escaping protections and injects the supplied content directly into the browser DOM.

An authenticated administrator — or an attacker who compromises administrative access — may inject malicious HTML or JavaScript event-handler payloads into the `keyTakeaway` field. The payload is stored persistently within the database and executes automatically in the browser of every client visiting the affected landing page.

Step to Reproduce

1. Review the landing-page creation handler within `server.js`.
2. Observe that surrounding fields are sanitised while `keyTakeaway` is stored directly from user-controlled input.
sectionType: validators.sanitizeHtml(section.sectionType, 100),
sectionTitle: validators.sanitizeHtml(section.sectionTitle, 300),
keyTakeaway: section.keyTakeaway || null,
3. Review the landing-page update handler and observe that the same unsanitised assignment is repeated.
keyTakeaway: section.keyTakeaway || null,
4. Review the public endpoint serving landing-page data.
keyTakeaway: section.keyTakeaway,

5. Review frontend React components rendering the supplied value using dangerouslySetInnerHTML.


```
<div dangerouslySetInnerHTML={{ __html: keyTakeaway }} />
```
6. Observe that the same rendering pattern exists across multiple frontend components:
 - BrandingSlide.jsx*
 - Social_Media.jsx*
 - TrafficShare.jsx*
 - MarketingChannels.jsx*
 - GapAnalysisDashboard.jsx*
 - GoogleReviewsChart.jsx*
 - HealthDashboard.jsx*
 - Pages_Analysis.jsx*
7. A malicious payload inserted into keyTakeaway therefore executes automatically when affected landing pages are viewed by clients.

Evidence

```
// Create landing page with validated sections
const landingPage = await prisma.customerLandingPage.create({
  data: {
    slug,
    clientName: validators.sanitizeHtml(clientName, 200),
    companyName: validators.sanitizeHtml(companyName, 200),
    date: parsedDate,
    heroTitle: validators.sanitizeHtml(heroTitle || 'SEO & GROWTH PROPOSAL FOR {Company Name}', 500),
    heroSubtitle: validators.sanitizeHtml(heroSubtitle || 'Data-driven strategy tailored to outperform your competitors', 500),
    sections: {
      create: sectionsData.map((section, index) => ({
        sectionType: validators.sanitizeHtml(section.sectionType, 100),
        sectionTitle: validators.sanitizeHtml(section.sectionTitle, 300),
        contentType: section.hasImage || fileMap[section.sectionType] ? 'image' : 'text',
        imagePath: fileMap[section.sectionType] || null,
        keyTakeaway: section.keyTakeaway || null,
        additionalData: validators.validateSectionData(section.sectionType, section.additionalData),
        order: index + 1 // Always use sequential order based on array position
      }))
    }
  },
  include: {
    sections: true
  }
})
```

Figure 14: The landing-page creation and update handlers store keyTakeaway directly from user-controlled input without applying validators.sanitizeHtml().

```
/* Key Takeaway Section */
<div className="container mx-auto pb-6 sm:pb-10 md:pb-16 text-center">
  <h2 className="text-xl md:text-2xl font-black text-white mb-6 lg:mb-12 uppercase">
    Key takeaway
  </h2>
  <div className="text-base text-white [&_h3]:text-lg [&_h3]:font-bold [&_h3]:mb-3 [&_h4]:text-lg [&_h4]:font-bold [&_h4]:mb-3 [&_p]:mb-3 [&_ul]:list-disc [&_ul]:text-left [&_ul]:max-w-3xl [&_ul]:mx-auto [&_li]:mb-2">
    <div dangerouslySetInnerHTML={{ __html: keyTakeaway }} />
  </div>
  <div className="space-y-8">
    <div>
      <h3 className="text-xl font-bold text-white mb-3">
        Client (Great Lakes Electric):
      </h3>
      <p className="text-white text-lg leading-relaxed">
        Demonstrates steady traffic growth throughout the period,
        reflecting improving online visibility.
      </p>
    </div>
    <div>
      <h3 className="text-xl font-bold text-white mb-4">
        Competitors:
      </h3>
      <ul className="space-y-4 text-white text-lg">
        <li>
          <span className="text-white font-bold"></span>{" "}
          <strong className="text-white">Competitor 1</strong> shows a
          mid-period traffic peak followed by a decline.
        </li>
        <li>

```

Figure 15: Frontend React components render the supplied keyTakeaway value using dangerouslySetInnerHTML, bypassing React's built-in XSS protections.

Business Impact

- An attacker with administrative access may inject persistent malicious HTML or JavaScript payloads into client-facing landing pages through the keyTakeaway field.
- The injected payload executes automatically in the browser of every client visiting the affected landing page and may be used to:
- manipulate page content,
- redirect clients to attacker-controlled infrastructure,
- harvest client information,
- capture selected service/package details,
- perform phishing attacks,
- or execute unauthorized actions within the client browser context.
- Because the payload is stored persistently within the database, the malicious content remains active until manually removed from affected landing-page records.

Remediation

It is recommended to apply consistent sanitisation controls to the keyTakeaway field using the existing `validators.sanitizeHtml()` functionality already implemented for adjacent fields.

The application should:

- sanitise all user-supplied HTML content before database storage,
- apply contextual output encoding before frontend rendering,
- avoid rendering untrusted content directly through `dangerouslySetInnerHTML`,
- and implement frontend sanitisation controls such as `DOMPurify` before DOM insertion.

4.5. Payment Access Tokens for All Proposals Logged to Application Startup Logs

Risk	Medium	Likelihood	Medium	CVSS Score	6.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N				
Category	Security Misconfiguration / OWASP A05				
Affected Assets	Payment Access Token Workflow / Application Startup Logging				

Description

During the source code security review, it was identified that the application automatically generates and logs valid payment access tokens for all proposals during every server startup.

The self-invoking function `seedStaticProposalTokens()` iterates through all proposal entries and prints fully functional payment URLs containing valid HMAC-signed access tokens directly to stdout using `console.log()`.

The generated tokens remain valid for 30 days and are accepted by the payment validation workflow implemented within the application.

No environment restriction or `NODE_ENV` validation was identified to prevent this behavior in production deployments. As a result, valid payment links may become exposed through centralized deployment logs, application logging systems accessible by developers, operators, or support personnel.

Step to Reproduce

1. Review the startup token generation logic within `server.js`.
(function seedStaticProposalTokens() {
2. Observe that the application iterates through all proposals during startup.
for (const p of proposals) {
3. Observe that payment access tokens are automatically generated.
const token = generatePaymentAccessToken(p.id);
4. Observe that fully functional payment URLs are printed directly to stdout.
console.log(`/p/\${p.id}?token=\${token}`);
5. Observe that no production environment restriction or debug-only guard is implemented.
6. Review payment token validation logic and confirm that logged tokens are accepted as valid payment access tokens.

Evidence

```
// Generate access tokens for proposals that were loaded statically from
// proposals.js (pre-existing test/seed data). These tokens are logged once
// on startup so developers can access them.
(function seedStaticProposalTokens() {
  const staticIds = proposals.map(p => p.id);
  if (staticIds.length > 0) {
    console.log('?? Payment access tokens for static proposals (valid 30 days):');
    for (const p of proposals) {
      const token = generatePaymentAccessToken(p.id);
      console.log(`  /p/${p.id}?token=${token}`);
    }
  }
})();
```

Figure 16: Application automatically generates payment access tokens for all proposals during server startup.

```
// Generate access tokens for proposals that were loaded statically from
// proposals.js (pre-existing test/seed data). These tokens are logged once
// on startup so developers can access them.
(function seedStaticProposalTokens() {
  const staticIds = proposals.map(p => p.id);
  if (staticIds.length > 0) {
    console.log('?? Payment access tokens for static proposals (valid 30 days):');
    for (const p of proposals) {
      const token = generatePaymentAccessToken(p.id);
      console.log(`  /p/${p.id}?token=${token}`);
    }
  }
})();
```

Figure 17: No production environment restriction identified before logging valid payment access URLs.

Business Impact

- Any user with access to application logs, logging systems may obtain valid payment access URLs for all proposals processed by the application.
- Successful abuse of exposed payment links may allow unauthorized access to proposal payment pages, exposure of customer payment-related information, unauthorized payment activity, and bypass of the intended controlled payment-link distribution workflow.

Remediation

- It is recommended to remove all sensitive token generation and payment-link logging functionality from production application startup workflows.
- Payment access tokens should only be generated when explicitly required and should never be printed to application logs, deployment logs, or stdout output.
- If startup token generation is required for development purposes, the functionality should be strictly restricted to non-production environments using appropriate `NODE_ENV` validation controls.

Additionally:

- rotate `TOKEN_SECRET` if exposure is suspected,
- minimize token lifetime where feasible,
- restrict log access permissions,
- and ensure sensitive authentication or payment tokens are excluded from all centralized logging systems.

4.6. TLS Certificate Verification Disabled on Admin SMTP Transport

Risk	Medium	Likelihood	Medium	CVSS Score	4.9
CVSS Vector	CVSS:3.1/AV:N/AC:H/PR:H/UI:N/S:U/C:H/I:N/A:N				
Category	Cryptographic Failures / OWASP A02				
Affected Assets	Administrative Email Notification System / SMTP Communication Channel				

Description

During the source code security review, it was identified that the application explicitly disables TLS certificate verification within the SMTP transport configuration used for administrative email communication.

The `createAdminEmailTransporter()` function configures Nodemailer using the following insecure TLS configuration:

```
tls: { rejectUnauthorized: false },
```

This setting disables X.509 certificate validation during the SMTP TLS handshake and causes the application to trust any certificate presented by the SMTP server, including self-signed or attacker-controlled certificates.

Step to Reproduce

1. Review the SMTP transporter initialization function within `server.js`.
function createAdminEmailTransporter() {
2. Observe that Nodemailer transport configuration is initialized.
return nodemailer.createTransport({
3. Review the TLS configuration section.
tls: { rejectUnauthorized: false },
4. Observe that certificate validation is explicitly disabled.
5. Review the OTP delivery workflow using the same transporter.
await transporter.sendMail(mailOptions);
6. An attacker controlling the SMTP network path may therefore intercept OTP emails using forged or self-signed TLS certificates.

Evidence

```
// Helper: create nodemailer transporter from env SMTP settings
function createAdminEmailTransporter() {
  const host = process.env.SMTP_HOST || process.env.EMAIL_HOST || 'smtp.gmail.com';
  const port = parseInt(process.env.SMTP_PORT || process.env.EMAIL_PORT) || 587;
  const user = process.env.SMTP_USER || process.env.EMAIL_USER;
  const pass = process.env.SMTP_PASS || process.env.EMAIL_PASS;
  if (!user || !pass) return null;
  return nodemailer.createTransport({
    host, port,
    secure: port === 465,
    auth: { user, pass },
    tls: { rejectUnauthorized: false },
  });
}
```

Figure 18: Application initializes administrative SMTP transport using Nodemailer.

```
// Helper: create nodemailer transporter from env SMTP settings
function createAdminEmailTransporter() {
  const host = process.env.SMTP_HOST || process.env.EMAIL_HOST || 'smtp.gmail.com';
  const port = parseInt(process.env.SMTP_PORT || process.env.EMAIL_PORT) || 587;
  const user = process.env.SMTP_USER || process.env.EMAIL_USER;
  const pass = process.env.SMTP_PASS || process.env.EMAIL_PASS;
  if (!user || !pass) return null;
  return nodemailer.createTransport({
    host, port,
    secure: port === 465,
    auth: { user, pass },
    tls: { rejectUnauthorized: false },
  });
}
```

Figure 19: TLS certificate verification explicitly disabled through `rejectUnauthorized: false`.

Business Impact

- An attacker positioned on the network path between the application server and the SMTP provider may intercept sensitive administrative email communications by presenting forged or attacker-controlled TLS certificates trusted by the application.
- Successful exploitation may allow interception of OTP verification codes, password-change notifications, and administrative email communications, potentially enabling unauthorized administrative account takeover and compromise of sensitive platform functionality.

Remediation

- It is recommended to remove the insecure TLS override and enforce strict certificate validation for all SMTP communications.

The application should:

- remove `rejectUnauthorized: false`,
- rely on Nodemailer's default secure TLS validation behavior,
- validate SMTP certificates using trusted Certificate Authorities (CAs),
- and avoid globally disabling certificate verification.
- If self-signed certificates are required for internal SMTP infrastructure, the trusted CA certificate should be explicitly configured instead of disabling verification globally.

4.7. Payment Replay Protection Uses Fail-Open Logic During Database Errors

Risk	Medium	Likelihood	Medium	CVSS Score	5.9
CVSS Vector	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:H/A:N				
Category	Business Logic Vulnerability / OWASP A04 – Insecure Design				
Affected Assets	Proposal Payment Workflow / Duplicate Payment Protection Logic				

Description

During the source code security review, it was identified that the application's duplicate-payment protection mechanism implements fail-open behavior during database errors.

The function `isProposalPaid()` is responsible for validating whether a proposal has already been paid before allowing payment processing to continue. However, when a database error occurs, the function catches the exception and explicitly returns `false`, treating the proposal as unpaid.

```
return false; // fail-open to avoid blocking legitimate flows on DB error
```

This behavior allows payment processing to continue during database outages, query failures, connection pool exhaustion, or transient Prisma exceptions.

The affected validation function is used across multiple critical payment-processing routes, including:

- proposal payment page rendering,
- payment intent creation,
- and subscription creation workflows.

As a result, previously paid proposals may be processed again during database failures, potentially resulting in duplicate Stripe charges.

Step to Reproduce

1. Review the duplicate-payment validation function within `server.js`.
async function isProposalPaid(proposalId) {
2. Observe that the application queries the `PaidProposal` table.
const row = await prisma.paidProposal.findUnique({
3. Review the exception handling logic.
catch (err) {
4. Observe that database errors explicitly return `false`.
return false;
5. Review the payment-processing routes using the same validation function.
if (proposalId && await isProposalPaid(proposalId)) {
6. During a database outage or query failure, the validation incorrectly treats previously paid proposals as unpaid and allows payment processing to continue.

Evidence

```
/**
 * Returns true if the proposal has already been paid (DB-backed).
 */
async function isProposalPaid(proposalId) {
  if (!proposalId) return false;
  try {
    const row = await prisma.paidProposal.findUnique({ where: { proposalId } });
    return row !== null;
  } catch (err) {
    console.error(`isProposalPaid DB error for "${proposalId}":`, err.message);
    return false; // fail-open to avoid blocking legitimate flows on DB error
  }
}
```

Figure 20: Application checks proposal payment status using the `isProposalPaid()` validation function.

```
/**
 * Returns true if the proposal has already been paid (DB-backed).
 */
async function isProposalPaid(proposalId) {
  if (!proposalId) return false;
  try {
    const row = await prisma.paidProposal.findUnique({ where: { proposalId } });
    return row !== null;
  } catch (err) {
    console.error(`isProposalPaid DB error for "${proposalId}":`, err.message);
    return false; // fail-open to avoid blocking legitimate flows on DB error
  }
}
```

Figure 21: Database errors are handled using fail-open logic that returns false instead of blocking payment processing.

Business Impact

- During database outages, connection failures, or transient query errors, the application may incorrectly treat previously paid proposals as unpaid and allow duplicate Stripe charges to be created.

Successful exploitation or operational failure may result in:

- duplicate client charges,
- financial disputes and chargebacks,
- inconsistent payment records,
- reputational damage,
- and financial liability resulting from unauthorized repeated payment processing.
- Because the affected logic protects critical payment workflows, fail-open behavior significantly weakens the integrity of the payment-processing system.

Remediation

- It is recommended to implement fail-closed behavior for all payment validation workflows.

The application should:

- treat database validation failures as temporary payment-processing errors,
- block payment processing during validation uncertainty,
- return appropriate HTTP 503 responses during database failures,
- and prevent payment continuation unless proposal payment status is successfully verified.

4.8. Hardcoded Development/Test Stripe Identifiers and Email Data in Repository Files

Risk	Medium	Likelihood	Medium	CVSS Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N				
Category	Sensitive Data Exposure / OWASP A02 – Cryptographic Failures				
Affected Assets	Development/Test Proposal Data / Source Code Repository				

Description

During the source code security review, it was identified that the OTP confirmation endpoint used for administrative credential changes does not implement any rate-limiting, retry restriction, lockout protection, or brute-force mitigation controls.

During the source code security review, it was identified that multiple repository files contain hardcoded development and operational data, including Stripe customer identifiers, Stripe subscription identifiers, and email addresses directly embedded within source-controlled files.

The exposed information was identified within:

- fix-refund-records.js
- test-refund-api.js
- proposals.js

The identified values appear to be related to development or operational testing activities; however, the information remains committed directly within repository files instead of being externalized through environment variables or sanitized test data.

Step to Reproduce

1. Review the file fix-refund-records.js.
const custId = 'cus_UM9Usdv003ZEyE';
2. Observe that Stripe subscription identifiers are hardcoded directly within repository scripts.
stripeSubscriptionId: 'sub_1TNRBVD60jTqpzFUSTEQc2uI'
3. Review the file test-refund-api.js.
stripeCustomerId: 'cus_ULjK7dQb5zh83n'
4. Review the file proposals.js.
email: 'shariarhosain1315@gmail.com'
5. Observe that development/test operational data is committed directly into source-controlled files.

Evidence

```
(async () => {  
  // Find the ACTUAL refunded charge in Stripe  
  const custId = 'cus_UM9Usvd003ZEyE';  
  const charges = await stripe.charges.list({ customer: custId, limit: 100 });  
  
  const refundedPis = new Set();  
  for (const ch of charges.data) {  
    if (ch.amount_refunded > 0) {  
      const piId = typeof ch.payment_intent === 'string' ? ch.payment_intent : ch.payment_intent?.id;  
      if (piId) refundedPis.add(piId);  
      console.log(`Refunded charge: ${ch.id} PI=${piId} amount=${ch.amount / 100} refunded=${ch.amount_refunded / 100}`);  
    }  
  }  
  console.log(`\nActually refunded PIs:`, [...refundedPis]);  
})
```

Figure 22: Stripe customer and subscription identifiers hardcoded within repository utility scripts

Business Impact

- Hardcoded operational and testing data within repository files may increase the risk of accidental information disclosure, insecure development practices, and internal operational misuse.

Remediation

- It is recommended to remove hardcoded operational identifiers and email data from repository files and replace them with anonymized placeholders or securely managed configuration inputs.

4.9. Application Allows Runtime Modification of Sensitive Environment Configuration

Risk	Medium	Likelihood	Medium	CVSS Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:L/I:H/A:L				
Category	Security Misconfiguration / Secure Configuration Management Weakness				
Affected Assets	Environment Configuration Management / Runtime Secret Handling Logic				

Description

During the source code security review, it was identified that the application contains functionality capable of modifying environment configuration values and sensitive operational settings during runtime.

The application programmatically updates both:

- environment configuration files (.env),
- and runtime environment variables (process.env).

The following operations were identified within the application logic:

```
fs.writeFileSync(envPath, envContent, 'utf8');
```

and:

```
process.env.SMTP_PASS = safePass;
```

This behavior introduces configuration integrity and operational security risks because sensitive secrets and infrastructure configuration should typically be managed externally through secure configuration management or secrets-management solutions rather than dynamically modified by application-layer code.

Step to Reproduce

1. Review the backend source code responsible for SMTP and administrative configuration handling.
2. Navigate to configuration update logic within: routes/payment-tracking.js
3. Observe file write operations modifying environment configuration content.

```
fs.writeFileSync(envPath, envContent, 'utf8');
```
4. Observe runtime modification of sensitive environment variables.

```
process.env.SMTP_PASS = safePass;  
process.env.SMTP_USER = safeUser;  
process.env.ADMIN_EMAIL = safeEmail;
```
5. Confirm that sensitive operational configuration values are dynamically modified by application-layer logic.

Evidence

```
fs.writeFileSync(envPath, envContent, 'utf8');

// Update process.env in-memory
if (safeEmail) process.env.ADMIN_EMAIL = safeEmail;
if (safeHost) process.env.SMTP_HOST = safeHost;
if (safePort) process.env.SMTP_PORT = safePort;
if (safeUser) process.env.SMTP_USER = safeUser;
if (safePass && safePass !== '*****') process.env.SMTP_PASS = safePass;

// Reset transporter so it picks up new settings
emailTransporter = null;
```

Figure 23: Application dynamically rewrites environment configuration content during runtime.

Business Impact

- If administrative functionality responsible for configuration management is abused or compromised, an attacker may modify sensitive operational settings and infrastructure-related configuration values directly from the application layer.

Remediation

- It is recommended to remove the capability for the application to modify environment configuration files and sensitive secrets during runtime.
- Sensitive configuration values should be managed through secure external configuration-management or secrets-management solutions instead of application-layer write operations.

4.10. Development and Operational Utility Scripts Present in Main Repository

Risk	Medium	Likelihood	Medium	CVSS Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:L/I:H/A:L				
Category	Security Misconfiguration				
Affected Assets	Source Code Repository / Internal Utility Scripts				

Description

During the source code security review, multiple development, testing, and operational utility scripts were identified within the primary application repository.

The identified scripts contain functionality related to:

- refund processing,
- synchronization operations,
- payment testing,
- administrative maintenance,
- and operational troubleshooting activities.

Examples identified include:

- test-refund-api.js
- test-sync.js
- fix-refund-records.js
- fix-refund-status.js
- test-f03.js

Although these scripts appear intended for internal operational or development use, storing such tooling within the primary production repository may increase operational risk if the files are accidentally deployed, executed, exposed, or reused within unintended environments.

Step to Reproduce

1. Review the repository file structure.
2. Identify development and operational utility scripts within the repository.
3. Observe scripts containing:
4. refund-processing logic,
5. payment testing functionality,
6. synchronization utilities,
7. and administrative maintenance operations.
8. Verify that these files are stored directly within the primary application repository.

Evidence

```
test-refund-api.js  
test-sync.js  
fix-refund-records.js  
fix-refund-status.js  
test-f03.js
```

Figure 24: Development and operational utility scripts identified within the primary application repository.

Business Impact

- The presence of development and operational utility scripts within the primary repository may increase operational exposure and deployment risk if such files are accidentally deployed, executed, or exposed within production environments.
- Additionally, these scripts may reveal internal business workflows, administrative logic, or operational procedures that could assist internal reconnaissance or increase the likelihood of operational misuse and human error.

Remediation

- It is recommended to separate development, testing, and operational utility scripts from the primary production application repository wherever feasible.

4.11. Runtime SMTP Configuration Changes Allow Persistent Administrative Email Redirection

Risk	Low	Likelihood	Medium	CVSS Score	3.7
CVSS Vector	CVSS:3.1/AV:N/AC:H/PR:H/UI:N/S:U/C:H/I:N/A:N				
Category	Security Misconfiguration				
Affected Assets	Administrative Email Configuration / Administrative Notification System				

Description

During the source code security review, it was identified that the application exposes an authenticated administrative API endpoint capable of modifying SMTP configuration values during runtime.

The endpoint:

- PUT /api/admin/settings/email

allows authenticated administrators to update SMTP host, username, password, and administrative email settings dynamically. The application applies the updated values directly into:

- runtime environment variables (process.env),
- and persistent .env configuration files.

Additionally, the application resets the active email transporter instance, causing all future administrative emails to use the updated SMTP configuration immediately.

Step to Reproduce

1. Review the SMTP configuration update endpoint within routes/payment-tracking.js.
`router.put('/api/admin/settings/email'`
2. Observe that SMTP configuration values are accepted from authenticated requests.
3. Review runtime environment modification logic.
`process.env.SMTP_HOST = safeHost;`
`process.env.SMTP_USER = safeUser;`
`process.env.SMTP_PASS = safePass;`
4. Observe that the updated configuration is persisted to the .env file.
`fs.writeFileSync(envPath, envContent, 'utf8');`
5. Observe that the email transporter is reset dynamically.
`emailTransporter = null;`
6. Confirm that future administrative emails use the updated SMTP configuration.

Evidence

```
* Update admin email settings
*/
router.put('/api/admin/settings/email', authMiddleware, async (req, res) => {
  try {
    const { adminEmail, smtpHost, smtpPort, smtpUser, smtpPass } = req.body;

    // Validate inputs to prevent .env injection (no newlines, control chars)
    const envSafe = (val) => {
      if (!val || typeof val !== 'string') return null;
      // Strip newlines, carriage returns, and null bytes to prevent env var injection
      return val.replace(/[\r\n\u0000]/g, '').trim().slice(0, 500);
    };

    const safeEmail = envSafe(adminEmail);
    const safeHost = envSafe(smtpHost);
    const safePort = envSafe(smtpPort);
    const safeUser = envSafe(smtpUser);
    const safePass = envSafe(smtpPass);

    if (safeEmail && !validators.isValidEmail(safeEmail)) {
      return res.status(400).json({ error: 'Invalid email format' });
    }
    if (safePort && (isNaN(parseInt(safePort)) || parseInt(safePort) < 1 || parseInt(safePort) > 65535)) {
      return res.status(400).json({ error: 'Invalid SMTP port' });
    }
  }
});
```

Figure 25: Authenticated administrative endpoint allows runtime modification of SMTP configuration values.

Business Impact

- An attacker with administrative access may modify SMTP configuration values and redirect future OTP emails, password reset notifications, and administrative communications to attacker-controlled mail infrastructure.
- Because the configuration changes are persisted to the .env file, the modified SMTP behavior may survive application restarts and create persistent post-compromise access to sensitive administrative email workflows.

Remediation

- It is recommended to remove the ability to modify SMTP infrastructure configuration through runtime application endpoints.
- Sensitive SMTP configuration values should be managed externally through secure infrastructure configuration or secrets-management platforms rather than dynamically modified through application-layer APIs.

Appendix 1: WAHH testing checklist

Test	Checked*	Vulnerable
Authentication & Session Management		
Review login and authentication workflow implementation	Yes	X
Review OTP verification and brute-force protection controls	Yes	Vulnerable
Review password complexity enforcement mechanisms	Yes	Vulnerable
Review duplicate password-management routes and logic consistency	Yes	Vulnerable
Review session handling and session invalidation mechanisms	Yes	X
Review administrative authentication protection controls	Yes	X
Review secure handling of session cookies and authentication tokens	Yes	X
Review session expiration and timeout enforcement	Yes	X
Review MFA and additional verification controls for sensitive operations	Yes	X
Review protection against Cross-Site Request Forgery (CSRF) attacks	Yes	X
Review secure cookie attributes (HttpOnly, Secure, SameSite)	Yes	X
Authorization & Access Control		
Review administrative route protection mechanisms	Yes	X
Review authorization middleware implementation	Yes	X
Review privilege validation for sensitive operations	Yes	X
Review access-control enforcement across APIs	Yes	X
Review protection of administrative configuration functionality	Yes	Vulnerable
Review access restrictions for payment-management workflows	Yes	X
Input Validation & Injection Security		
Review backend input validation mechanisms	Yes	X
Review sanitization controls for user-controlled HTML content	Yes	Vulnerable
Review frontend rendering security controls	Yes	Vulnerable
Review protection against stored XSS vulnerabilities	Yes	Vulnerable
Review server-side rendering workflows for injection risks	Yes	Vulnerable

Review HTML and JavaScript injection protections	Yes	Vulnerable
Review PDF-generation workflows for unsafe rendering behavior	Yes	Vulnerable
Review output encoding controls across frontend components	Yes	Vulnerable
Review file upload validation and content-type enforcement	Yes	X
Review server-side external request handling for SSRF exposure	Yes	X
Frontend Security		
Review React component rendering security	Yes	Vulnerable
Review usage of dangerouslySetInnerHTML	Yes	Vulnerable
Review handling of untrusted client-facing content	Yes	Vulnerable
Review frontend content sanitization controls	Yes	Vulnerable
Review client-side rendering workflows for injection exposure	Yes	Vulnerable
Review implementation of HTTP security headers	Yes	X
Backend Application Security		
Review backend API security implementation	Yes	X
Review administrative API workflows	Yes	X
Review application error handling and fail-secure behavior	Yes	Vulnerable
Review server-side processing workflows	Yes	Vulnerable
Review application-layer configuration modification logic	Yes	Vulnerable
Review operational workflow protections	Yes	X
Payment & Business Logic Security		
Review payment-processing workflow security	Yes	X
Review payment replay protection mechanisms	Yes	Vulnerable
Review payment access token generation and validation logic	Yes	X
Review payment token exposure within operational logs	Yes	Vulnerable
Review Stripe integration security controls	Yes	X
Review business logic protections for proposal-payment workflows	Yes	X
Secrets & Configuration Management		
Review runtime environment variable handling	Yes	Vulnerable
Review runtime modification of .env configuration files	Yes	Vulnerable
Review secure handling of SMTP credentials	Yes	Vulnerable
Review secrets exposure risks within repository files	Yes	Vulnerable
Review configuration-management security practices	Yes	Vulnerable

Review secure separation of operational configuration from application logic	Yes	Vulnerable
Cryptography & Sensitive Data Protection		
Review password hashing implementation	Yes	X
Review token-signing mechanisms	Yes	X
Review SMTP TLS certificate validation	Yes	Vulnerable
Review secure transport protection for administrative communications	Yes	Vulnerable
Review sensitive data handling within operational workflows	Yes	X
Logging, Monitoring & Error Handling		
Review logging of sensitive operational information	Yes	Vulnerable
Review exposure of payment access tokens within logs	Yes	Vulnerable
Review error handling and exception-management logic	Yes	Vulnerable
Review audit logging for sensitive administrative operations	Yes	X
Review operational monitoring and security visibility controls	Yes	X
Repository & Secure SDLC Review		
Review repository hygiene and operational artifact exposure	Yes	Vulnerable
Review presence of development and utility scripts	Yes	Vulnerable
Review hardcoded development/test operational identifiers	Yes	Vulnerable
Review secure coding consistency across application modules	Yes	Vulnerable
Review separation of development and production operational tooling	Yes	Vulnerable
Third-Party & Dependency Security		
Review Stripe integration security implementation	Yes	X
Review third-party API usage and operational controls	Yes	X
Review external dependency handling practices	Yes	X
Review package dependencies for known vulnerable components	Yes	X
Review dependency version management and update practices	Yes	X
Review third-party package trust and supply-chain exposure	Yes	X
Review operational security of integrated email services	Yes	Vulnerable
Review third-party communication security controls	Yes	Vulnerable
Miscellaneous tests		
Review exposure of sensitive information through debug functionality	Yes	X
Review application startup routines for insecure operational behavior	Yes	Vulnerable
Review exposure of internal operational workflows within source code	Yes	Vulnerable

Review unsafe use of development-only functionality within production code	Yes	Vulnerable
Review runtime generation and exposure of sensitive operational tokens	Yes	Vulnerable
Review handling of temporary operational data and artifacts	Yes	X
Review consistency of security controls across application modules	Yes	Vulnerable
Review application behavior during backend/database failures	Yes	Vulnerable
Review fail-open vs fail-secure operational logic	Yes	Vulnerable
Review operational email notification workflows	Yes	Vulnerable
Review handling of administrative operational settings	Yes	Vulnerable
Review exposure of internal business logic through client-facing functionality	Yes	X
Review hardcoded operational or testing values within application files	Yes	Vulnerable
Review application configuration persistence mechanisms	Yes	Vulnerable
Review protection of sensitive administrative workflows	Yes	X

* Yes / No / Not Applicable

Appendix 2: Risk rating metrics

1.1. Vulnerability rating

The vulnerability rating is based on the CVSS v3.1 standard while evaluating only the base metric.

Please refer to <https://www.first.org/cvss/user-guide> for more details.

1.2. Risk rating evaluation

The grading is based on the risk associated with the vulnerability defined by its impact and by the likelihood of exploitability, and in some cases, based on the authentication and the location parameters.

For each vulnerability, an estimate of risk rating is defined by all the previous criteria using the following method:

► **Risk = Impact * Likelihood * (Authentication + Location)**

- Ex1: Risk1 = (Critical impact * Low likelihood * (Anonymous + distant) = C-High.
- Ex2: Risk2 = (High impact * Medium likelihood * (Authenticated + Local) = B-Medium.

Note: In some cases, the two criteria: **Authentication** and **Location** may not have any impact on the risk. E.g., If the application being tested is not accessible from the internet, the **location** parameter does not have any impact on the risk evaluation.

D-Critical	The risk associated with the vulnerability allows total control of the organization or a sensitive component (system, service, application).
C-High	The risk associated with the vulnerability allows total to partial control of the affected component with no critical overall impact, or disclosure of highly sensitive information.
B-Medium	The risk associated with the vulnerability allows information disclosure which could facilitate advanced and targeted attacks.
A-Low	The risk associated with the vulnerability is very limited. It usually represents a lack of good practices follow-up but may not have an impact on the company's stakes.

1.3. Remediation effort

For each vulnerability identified, an estimate is defined for the ease of correction to correct or limit the vulnerability. The following possible values are used: Complex, Moderate, and Low. Below is a detailed description of each case.

Complex	The correction of the identified vulnerability requires a complete modification of the code or configuration. This correction has an important impact on the system, or the application tested.
Moderate	The correction of the identified vulnerability requires some modification of the infrastructure supporting the application. Its impact needs to be identified to ensure the evasion of side effects.
Low	The correction of the identified vulnerability requires a simple modification that will have little to no impact on the system or the application tested.

Appendix 3: Testing methodology

The assessment was conducted using a manual gray-box Source Code Security Review approach, combining secure code analysis, application security review, business logic validation, and controlled vulnerability verification techniques.

The methodology is aligned with:

- OWASP Application Security Verification Standard (ASVS)
- OWASP Top 10 – Web & API Security Risks
- OWASP Secure Code Review Guide
- OWASP Web Security Testing Guide (WSTG)
- Penetration Testing Execution Standard (PTES)
- Secure Software Development Lifecycle (Secure SDLC) Best Practices
- CWE/SANS Top 25 Software Errors

Testing was performed using authenticated application access and GitLab repository access provided for the source code review engagement. The assessment included manual source code review, secure logic analysis, vulnerability validation, and controlled proof-of-concept testing where applicable.

You will find below a preview (the listed items are not exhaustive) of tests performed for each area:

▶ **W1 – Application Architecture & Attack Surface Review**

- Identification of application components, backend APIs, and frontend rendering workflows
- Review of application routing structure and exposed functionality
- Identification of administrative workflows and sensitive business logic
- Review of third-party integrations and external service dependencies
- Mapping of application trust boundaries and sensitive processing flows

▶ **W2 – Authentication & Session Management Security**

- Review of login and authentication workflow implementation
- Assessment of OTP verification and brute-force protections
- Review of password-management functionality and complexity enforcement
- Validation of session handling, invalidation, and authentication-token security
- Assessment of administrative authentication protection mechanisms

▶ **W3 – Authorization & Access Control Review**

- Review of authorization middleware and access-control enforcement
- Validation of protection mechanisms for administrative functionality
- Assessment of privilege validation for sensitive operations
- Review of API access restrictions and protected application routes

- Testing for unauthorized access and workflow misuse scenarios
- ▶ **W4 – Input Validation & Injection Security**
 - Review of backend input-validation controls
 - Assessment of frontend rendering security mechanisms
 - Testing for stored and reflected injection vulnerabilities
 - Review of sanitization and output-encoding controls
 - Assessment of server-side rendering workflows and unsafe HTML rendering behavior
- ▶ **W5 – Frontend & Client-Side Security Review**
 - Review of React component rendering behavior
 - Assessment of dangerouslySetInnerHTML usage and unsafe DOM rendering
 - Validation of client-side content sanitization mechanisms
 - Review of client-facing rendering workflows for injection exposure
 - Assessment of frontend handling of untrusted content
- ▶ **W6 – Backend Application Security Review**
 - Review of backend API security implementation
 - Assessment of server-side business logic and workflow protections
 - Validation of fail-secure and exception-handling mechanisms
 - Review of operational processing workflows and administrative APIs
 - Assessment of runtime configuration modification logic
- ▶ **W7 – Payment & Business Logic Security**
 - Review of payment-processing workflows and proposal-payment functionality
 - Assessment of payment replay protections and validation logic
 - Review of payment token generation and validation mechanisms
 - Validation of payment workflow integrity and fail-secure controls
 - Assessment of Stripe integration security implementation
- ▶ **W8 – Secrets & Configuration Management Review**
 - Review of runtime environment-variable handling
 - Assessment of runtime modification of sensitive operational configuration
 - Review of SMTP credential handling and operational email workflows
 - Identification of hardcoded operational identifiers and sensitive repository artifacts
 - Assessment of secure separation between operational configuration and application logic

▶ **W9 – Cryptography & Sensitive Data Protection**

- Review of password hashing and token-signing mechanisms
- Assessment of SMTP TLS certificate validation controls
- Validation of secure transport protection for sensitive communications
- Review of sensitive data handling within application workflows
- Assessment of token generation and validation security controls

▶ **W10 – Logging, Monitoring & Error Handling**

- Review of logging of sensitive operational information
- Assessment of exposure of sensitive tokens and operational data within logs
- Validation of audit logging for sensitive administrative operations
- Review of exception handling and operational error-management logic
- Assessment of security visibility and operational monitoring controls

▶ **W11 – Repository & Secure SDLC Review**

- Review of repository hygiene and operational artifact exposure
- Assessment of development and operational utility scripts within the repository
- Review of secure coding consistency across application modules
- Assessment of separation between development and production operational tooling
- Review of secure software development lifecycle practices

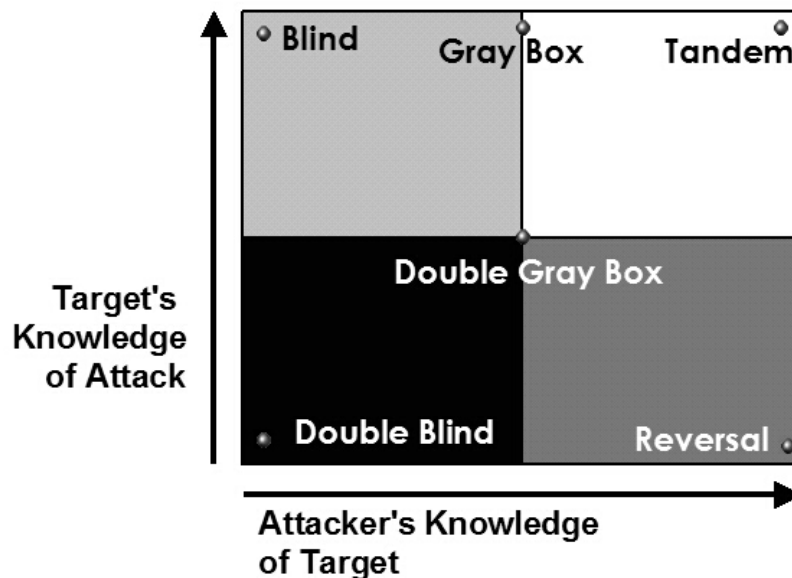
▶ **W12 – Third-Party & Dependency Security Review**

- Review of Stripe integration security controls
- Assessment of third-party API usage and communication security
- Review of dependency-management and external package handling practices
- Assessment of operational security of integrated email services
- Review of supply-chain and third-party operational exposure risks

1.4. Testing Types

The six types of testing described below are different depending on the amount of information that the tester has on the target and the knowledge possessed by the target on the tester.

As part of the mission, the purpose of this report, a specific methodology adapted to the context has been implemented inspired from several elements described below.



► **Blind**

- The tester undertakes the testing on the target with no prior knowledge of the security system, its components, or its flows. The target is prepared to be tested while knowing in advance the details of this exercise.

► **Double-Blind**

- The tester undertakes the testing on the target with no prior knowledge of the security system, its components, or its flows. The target is not notified in advance of the scope of this exercise. Channels and vectors used for testing are also not announced.

► **Grey Box**

- The tester undertakes the testing on target with limited knowledge of the security system, its components, and flows. The target is prepared to be tested while knowing in advance the details of this testing.

► **Double Grey Box**

- The tester undertakes the testing on targets with limited knowledge of the security system, its components and has complete visibility of flows. The target is prepared to be tested while knowing in advance the details of this exercise.

► **Tandem**

- The tester and the target are prepared for the testing and know in advance all the details of the exercise.

► **Reversal**

- The tester undertakes a review of the target with full knowledge of all its processes and its operational security system. On the other hand, the target has no information on the nature of the testing exercises or the date of their launches, and how they are realized.

Application Security & Secure Code Review Standards

Reference:

- OWASP Application Security Verification Standard (ASVS)
 - <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP Web Security Testing Guide (WSTG)
 - <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP Top 10 – Web Application Security Risks
 - <https://owasp.org/www-project-top-ten/>
- OWASP API Security Top 10
 - <https://owasp.org/www-project-api-security/>
- OWASP Secure Code Review Guide
 - <https://owasp.org/www-project-code-review-guide/>
- Penetration Testing Execution Standard (PTES)
 - <http://www.pentest-standard.org/>
- CWE/SANS Top 25 Most Dangerous Software Weaknesses
 - https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html
- NIST Secure Software Development Framework (SSDF)
 - <https://csrc.nist.gov/projects/ssdf>
- Secure Software Development Lifecycle (Secure SDLC) Best Practices
 - <https://owasp.org/www-project-samm/>

Source Code Security Review

The assessment was conducted in alignment with industry-recognized application security and secure software development practices, specifically addressing:

Secure Application Architecture & Configuration Review

Validation of secure application design, authentication mechanisms, authorization controls, secure configuration handling, payment workflow protections, and frontend/backend security implementation practices.

Vulnerability Identification

Manual identification of security vulnerabilities through source code review, business logic analysis, authentication and session-management review, injection testing, configuration analysis, and controlled proof-of-concept validation techniques.

Vulnerability Remediation

Provision of actionable remediation guidance, prioritization based on risk severity, and recommendations aligned with secure coding standards, secure configuration practices, and secure software development lifecycle principles.